

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 3**



Single Linked List Circular dan Double Linked List Circular

Oleh:

Rahma Syarita Jaya NIM. 2510817320004

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 3

Laporan Praktikum Algoritma & Struktur Data Modul 3: Single Linked List Circular dan Double Linked List Circular ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Rahma Syarita Jaya
NIM : 2510817320004

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
SOAL 1: Implementasi Senarai Berantai Tunggal Melingkar	6
A. Source Code.....	6
B. Pembahasan	10
SOAL 2: Si Palui dan Mesin Hitung Aneh	13
A. Source Code.....	14
B. Output Program	15
C. Pembahasan	15
SOAL 3: Implementasi Queue	17
A. Source Code.....	17
B. Pembahasan	21
SOAL 4: Si Palui dan Deretan Angka.....	23
A. Source Code.....	24
B. Output Program	25
C. Pembahasan	25
TAUTAN GIT	27

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Jawaban Soal 1	15
Gambar 2 Screenshot Hasil Jawaban Soal 2	25

DAFTAR TABEL

Tabel 1 Source code single_linked_list.cpp	6
Tabel 2 Source code prob1.cpp	14
Tabel 3 Source code double_linked-list.cpp.....	17
Tabel 4 Source code prob_b.cpp	24

SOAL 1: Implementasi Senarai Berantai Tunggal Melingkar

General Technical Requirements:

Standard: C++23

Memory Management: Manual (new / delete)

Constraints: Strictly NO STL (<vector>, <list>, dll), NO Global Arrays.

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 1 Source code single_linked_list.cpp

```
1  #include "single_linked_list.h"
2  #include <iostream>
3  #include <stdexcept>
4
5  void SingleLinkedList::init() {
6      head = new Node;
7      head->data = 0;
8      head->next = head;
9      tail = head;
10     size = 0;
11 }
12
13 bool SingleLinkedList::is_empty() {
14     return size == 0;
15 }
16
17 void SingleLinkedList::add_front(int val) {
18     Node* newNode = new Node;
19     newNode->data = val;
20     newNode->next = head->next;
21     head->next = newNode;
22
23     if(size == 0) {
```

```

24         tail = newNode;
25     }
26     size++;
27 }
28
29 void SingleLinkedList::add_back(int val){
30     Node* newNode = new Node;
31     newNode->data = val;
32     newNode->next = head;
33
34     tail->next = newNode;
35     tail = newNode;
36     size++;
37 }
38
39 void SingleLinkedList::add_idx(int val, int idx){
40     if (idx < 0 || idx > size) throw
std::out_of_range("Index tidak valid");
41     if (idx == 0) {
42         add_front(val);
43     } else if (idx == size) {
44         add_back(val);
45     } else {
46         Node* prev = head;
47         for (int i = 0; i < idx; i++) {
48             prev = prev->next;
49         }
50         Node* newNode = new Node;
51         newNode->data = val;
52         newNode->next = prev->next;
53         prev->next = newNode;
54         size++;
55     }
56 }
57
58 void SingleLinkedList::delete_front(){
59     if (is_empty()) return;
60
61     Node* temp = head->next;
62     head->next = temp->next;
63
64     if (size == 1) {
65         tail = head;
66     }
67     tail->next = head;
68     delete temp;
69     size--;

```

```

70 }
71
72 void SingleLinkedList::delete_back(){
73     if (is_empty()) return;
74
75     Node* prev = head;
76     while (prev->next != tail) {
77         prev = prev->next;
78     }
79     Node* temp = tail;
80     prev->next = head;
81     tail = prev;
82
83     delete temp;
84     size--;
85 }
86
87
88 void SingleLinkedList::delete_idx(int idx){
89     if (idx < 0 || idx >= size) throw
std::out_of_range("Index tidak valid");
90     if (idx == 0) {
91         delete_front();
92     } else if (idx == size - 1) {
93         delete_back();
94     }
95     else {
96         Node* prev = head;
97         for (int i = 0; i < idx; i++) {
98             prev = prev->next;
99         }
100
101         Node* temp = prev->next;
102         prev->next = temp->next;
103
104         delete temp;
105         size--;
106     }
107 }
108
109 void SingleLinkedList::display(){
110     if (is_empty()) {
111         std::cout << "List kosong\n";
112         return;
113     }
114     Node* curr = head->next;
115     while (curr != head) {

```



```

116         std::cout << curr->data << " ";
117         curr = curr->next;
118     }
119     std::cout << "\n";
120 }
121
122 int SingleLinkedList::get(int idx){
123     if (idx < 0 || idx >= size) throw
std::out_of_range("Index tidak valid");
124     Node* curr = head->next;
125     for (int i = 0; i < idx; i++) {
126         curr = curr->next;
127     }
128     return curr->data;
129 }
130
131
132 void SingleLinkedList::set(int val, int idx){
133     if (idx < 0 || idx >= size) throw
std::out_of_range("Index tidak valid");
134     Node* curr = head->next;
135     for (int i = 0; i < idx; i++) {
136         curr = curr->next;
137     }
138     curr->data = val;
139 }
140
141 void SingleLinkedList::clear(){
142     while (!is_empty()) {
143         delete_front();
144     }
145 }

```

B. Pembahasan

Inisialisasi fungsi:

1. `Init()`

berfungsi untuk menetapkan nilai awal pada variable agar tidak bernilai kosong, atau node pertama, elemen dasar yang menyimpan data dan pointer ke node lainnya. kompleksitas Waktu $O(1)$ karena hanya mengatur nilai beberapa variabel awal.

2. `is_empty()`

berfungsi untuk mengecek apakah list tersebut kosong. Kompleksitas Waktu $O(1)$ karena hanya mengecek satu variable.

3. `add_front()`

berfungsi untuk menambahkan data di posisi paling depan setelah Dummy Head atau simpul tambahan yang ditempatkan di bagian paling awal sebuah struktur data agar mudah dimanipulasi. Kalau list sebelumnya kosong, node baru ini akan menjadi tail. Kompleksitas Waktu $O(1)$ karena hanya memasukkan node baru. Kompleksitas Waktu $O(1)$ karena hanya mengecek satu variable.

4. `add_back()`

berfungsi untuk menambahkan data di posisi paling belakang dengan cara membuat node baru dan mengisi nilainya lalu memperbarui variabel tail ke node paling belakang yang baru. Kompleksitas Waktu tetap $O(1)$ karena satu kondisi if tidak membatalkan status kompleksitas konstan $O(1)$ dimana Status $O(1)$ baru akan gugur kalau ada perulangan.

5. `add_idx()`

berfungsi untuk menyisipkan data baru pada index dengan validasi if (`idx < 0 || idx > size`) untuk mengecek terlebih dahulu apakah index yang diminta sesuai dengan aturan. Jika tidak, program akan melempar pesan error(`throw std::out_of_range` dan mencegah program melanjutkan proses agar tidak terjadi crash Kondisi Awal if (`idx == 0`)): Jika targetnya adalah index atau paling depan, fungsi takan mendelegasikan tugas dengan memanggil fungsi `add_front(val)`.

6. `delete_front()`

berfungsi untuk menghapus node di index spesifik. Target yang sudah dihapus ini kemudian dihapus dari memori komputer atau delete tem).

7. `delete_front()`

berfungsi untuk menghapus data paling depan setelah head guna menghindari memory leaks. Dengan command `head->next = temp->next;`, panah dari head langsung berpindah ke node setelah temp. Fungsi juga memastikan jika node yang dihapus itu adalah

satu-satunya data. Karena temp sudah tidak masuk dalam rangkaian list, node ini segera dihancurkan dari Heap agar tidak terjadi memory leaks.

8. `delete_back()`

berfungsi untuk menghapus node di posisi paling ekor atau belakang guna menghindari memory leaks. Pertama fungsi juga akan mengecek apakah list kosong dengan `if is_empty()` lalu melakukan return agar tidak terjadi error. Jika syarat diluar daripada itu, karena ini adalah Single Linked List fungsi harus melakukan perulangan `while` untuk mencari node yang letaknya tepat sebelum ekor, tetapi aku lupa dengan command `prev->next = head;`, panah dari node prev langsung berpindah melingkar kembali ke head untuk memutus temp, dan prev diresmikan menjadi tail yang baru. Setelah itu, `delete temp;`. Karena temp sudah tidak masuk dalam rangkaian list, node ini segera dihancurkan dari Heap agar tidak terjadi memory leaks.

9. `delete_idx()`

berfungsi untuk menghapus node guna menghindari memory leaks. Pertama fungsi akan mengecek apakah indeks yang akan dihapus logis dengan `if (idx < 0 || idx >= size)` lalu melakukan throw karena indeks mulai dari 0 dan data yang diberikan ada 5 maka indeks maksimalnya ialah 4, jika melakukan delete idk ke-5 maka akan terjadi error. Jika indeks berada di urutan paling akhir maka fungsi akan memanggil `delete_back()` untuk menghapus ekor. contoh kasus, Jika syarat diluar daripada itu maka akan melakukan else dimana akan indeks yang akan dihapus akan ditaruh ke variable sementara atau temp.. lalu dengan command `prev->next = temp->next;` langsung berpindah ke node setelah temp. Setelah itu, `delete temp;` Karena temp sudah tidak masuk dalam rangkaian list, node ini segera dihancurkan dari Heap agar tidak terjadi memory leaks dan terakhir, ukuran size dikurangi.

10. `display()`

berfungsi untuk menampilkan seluruh isi data di dalam list ke terminal. Pertama fungsi akan mengecek apakah list tersebut sedang kosong dengan `if (is_empty())` dan mencetak teks "List kosong" lalu melakukan return agar program berhenti mengeksekusi sisa baris di bawahnya. Jika list ada isinya, fungsi akan membuat variabel penunjuk bernama curr atau current yang mulai berdiri di node pertama asli `head->next`. Karena list ini bersifat melingkar maka perulangan `while curr != head` akan terus berjalan mencetak nilai `curr->data`, sampai curr melingkar penuh dan kembali menyentuh head.

11. `get()`

berfungsi untuk mengambil atau membaca nilai pada posisi indeks tertentu tanpa menghapusnya dari list. Program menggunakan perulangan `for` untuk menyusuri list langkah demi langkah. Setelah berjalan sebanyak idx dan curr berhenti tepat di node yang diminta, fungsi langsung mengembalikan nilai dari node tersebut ke program utama dengan command `return curr->data;`.

12. `set()`

berfungsi untuk mengubah atau menimpa nilai data pada posisi (indeks) tertentu dengan nilai (val) yang baru. Jika indeksnya benar, penunjuk curr akan mulai dari head->next lalu menyusuri list menggunakan perulangan for sampai menemukan node target. Fungsi tidak mengembalikan nilai, melainkan langsung menimpa data lama di dalam node tersebut dengan data yang baru melalui command `curr->data = val;`.

13. `clear()`

berfungsi untuk membersihkan semua node yang ada di dalam list guna menghindari memory leaks. fungsi ini akan terus-menerus memanggil fungsi `delete_front()`; sampai list tersebut benar-benar kosong.

SOAL 2: Si Palui dan Mesin Hitung Aneh

Time Limit: 1.0 s Memory Limit: 64 MB
--

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh N buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh K langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ($K=K+1$).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ($K=K-1$).
- Jika pada suatu titik energi lompatan habis atau negatif ($K \leq 0$), sistem akan mengalami reset dan mengembalikan nilai lompatan ke nilai K awal (saat pancaran energi pertama kali diinisialisasi).

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

Batasan

- $1 \leq N \leq 104$
- $1 \leq K \leq 109$
- $1 \leq A_i \leq 106$ (Nilai ukiran pada batu)
- Program wajib menggunakan impleemntasi struktur data yang dibuat dari soal 1.

Keterangan

Format Input

N K

A_1 A_2 ... A_N

Format Output

Sebuah bilangan bulat tunggal yang merupakan nilai ukiran pada batu terakhir yang tersisa

Contoh Kasus

Input	Output
5 2 1 4 3 6 2	6
4 5 10 11 12 13	12

A. Source Code

Tabel 2 Source code prob_a.cpp

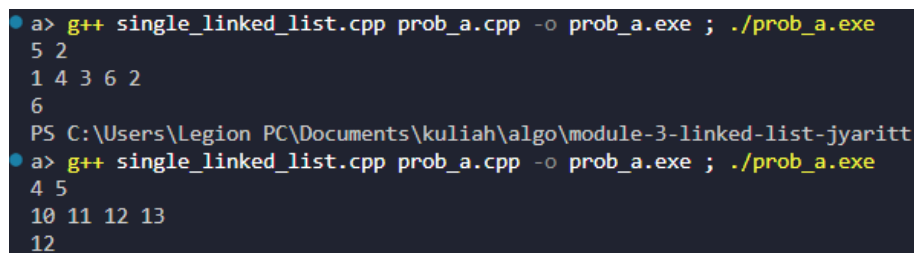
1	<code>#include "single_linked_list.h"</code>
2	<code>#include <iostream></code>
3	<code>using namespace std;</code>
4	
5	<code>int main() {</code>
6	<code> int n, k;</code>
7	<code> if (!(cin >> n >> k)) return 0;</code>
8	
9	<code> SingleLinkedList sll;</code>
10	<code>sll.init();</code>
11	
12	<code> for (int i = 0; i < n; i++) {</code>
13	<code> int val;</code>
14	<code> cin >> val;</code>
15	<code> sll.add_back(val);</code>
16	<code> }</code>
17	
18	<code> int k_awal = k;</code>
19	<code> int current_idx = 0;</code>
20	
21	<code> while (sll.size > 1) {</code>
22	<code> current_idx = (current_idx + k - 1) % sll.size;</code>
23	
24	<code> int batu_hancur = sll.get(current_idx);</code>
25	<code>sll.delete_idx(current_idx);</code>
26	
27	<code> if (batu_hancur % 2 == 0) {</code>
28	<code> k++;</code>
29	<code> } else {</code>
30	<code> k--;</code>
31	<code> }</code>
32	

```

33         if (k <= 0) {
34             k = k_awal;
35         }
36     }
37
38     cout << sll.get(0) << "\n";
39     sll.clear();
40     return 0;
41 }

```

B. Output Program



```

a> g++ single_linked_list.cpp prob_a.cpp -o prob_a.exe ; ./prob_a.exe
5 2
1 4 3 6 2
6
PS C:\Users\Legion PC\Documents\kuliah\algo\module-3-linked-list-jyaritt
a> g++ single_linked_list.cpp prob_a.cpp -o prob_a.exe ; ./prob_a.exe
4 5
10 11 12 13
12

```

Gambar 1 Screenshot Hasil Jawaban Soal 1

C. Pembahasan

Pada awal program, `cin >> n >> k` digunakan untuk membaca dua input dari pengguna, yaitu `n` sebagai jumlah total batu dan `k` sebagai jumlah langkah awal. Pengecekan `if (!...)` berfungsi untuk menghentikan program apabila input yang dimasukkan tidak valid atau kosong. Setelah itu, `sll.init();` dipanggil untuk menyiapkan struktur Single Linked List sebagai tempat data batu

Selanjutnya, program meminta input data batu melalui perulangan `for (int i = 0; i < n; i++)`. Setiap data batu yang dibaca dari input ditambahkan ke bagian belakang list menggunakan fungsi `sll.add_back(val)`. Selanjutnya ada `k_awal = k`; untuk menyimpan cadangan nilai langkah awal dan `current_idx = 0`; untuk menentukan titik awal perhitungan pada indeks pertama sebelum permainan dimulai.

Selama jumlah batu di dalam list masih lebih dari satu, seluruh logika permainan akan terus dieksekusi berulang kali. Program akan menentukan target batu yang akan dihancurkan menggunakan rumus `current_idx = (current_idx + k - 1) % sll.size;`.

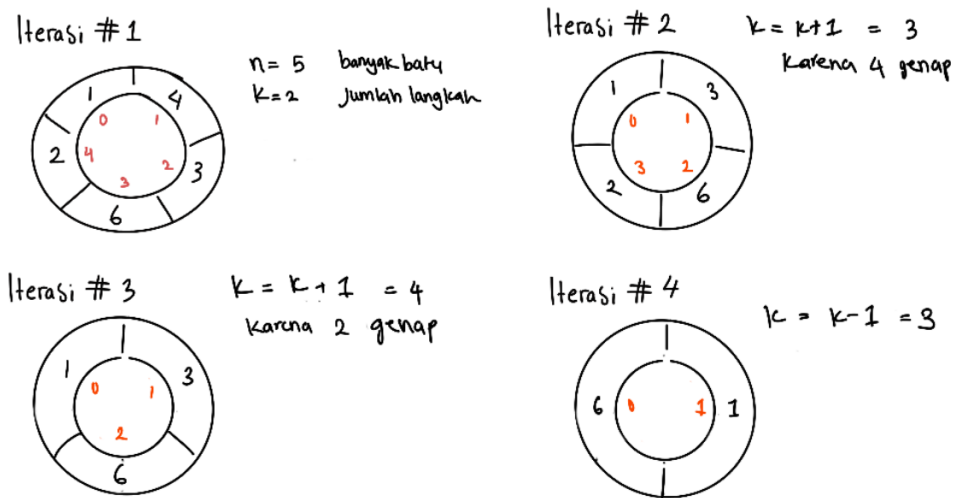
Setelah indeks ditemukan, `sll.get(current_idx);` mengambil dan menyimpan nilai sementara batu yang akan dihancurkan ke variabel `batu_hancur`, kemudian batu tersebut dihapus dari list menggunakan `sll.delete_idx(current_idx);`.

Berikutnya, program memperbarui aturan langkah dengan kondisi ganjil dan genap menggunakan modulo dan kondisi `if (batu_hancur % 2 == 0)`. Jika genap, jumlah langkah untuk putaran berikutnya akan bertambah satu, sedangkan jika ganjil maka Langkah akan

berkurang satu. Setelah itu, terdapat pengecekan if ($k \leq 0$) yang berfungsi sebagai mekanisme pengaman. Jika nilai k menjadi nol atau negatif akibat pengurangan sebelumnya, maka nilainya akan di-reset kembali ke k_awal .

Ketika perulangan while selesai, artinya hanya tersisa satu batu di dalam list. Program kemudian menampilkan batu yang tersisa tersebut melalui `cout << sll.get(0) << "\n";`. Fungsi `sll.clear();` membersihkan seluruh sisa memori agar terhindar dari memory leak.

Misalkan ambil contoh output pertama, berikut adalah ilustrasi iterasi atau proses perulangan suatu rangkaian instruksi.



Penjelasan iterasi:

Iterasi pertama, banyak batu adalah 5 dengan $[1, 4, 3, 6, 2]$ dan langkah batu ialah 2. Lakukan rumus $current_idx = (current_idx + k - 1) \% sll.size$; atau $current_idx = (0 + 2 - 1) \% 5 = 1$. Pada indeks 1, terdapat angka 4 dan batu dihancurkan. Selanjutnya, karena 4 adalah genap maka langkah akan bertambah satu menjadi 3, Lakukan $current_idx$ seperti yang sama maka akan mendapatkan nilai 3. Pada indeks ke-3 terdapat angka 2, lalu hancurkan,

Iterasi ketiga tersisa 3 angka, yakni $[1, 3, 6]$, karena tadi angka 2 adalah genap maka langkah akan bertambah menjadi 4. Hitung kembali $current_idx$ maka akan mendapat angka 0. Di gambar sendiri $current_idx = (4 + 2 - 1) \% 5 = 1$. Maka angka 1 pada indeks pertama akan dihapus. Menyisakan angka 6 yang bertahan sampai akhir. Logika ini dinamakan logika Josephus.

SOAL 3: Implementasi Senarai Berantai Ganda Melingkar

General Technical Requirements:

Standard: C++23

Memory Management: Manual (new / delete)

Constraints: Strictly NO STL (<vector>, <list>, dll), NO Global Arrays.

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList` beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 3 Source code double_linked-list.cpp

```
1  #include "double_linked_list.h"
2  #include <iostream>
3  #include <stdexcept>
4
5  void DoubleLinkedList::init() {
6      head = nullptr;
7      tail = nullptr;
8      size = 0;
9  }
10
11 bool DoubleLinkedList::is_empty() {
12     return size == 0;
13 }
14
15 void DoubleLinkedList::add_front(int val) {
16     Node* newNode = new Node;
17     newNode->data = val;
18
19     if (is_empty()) {
20         head = tail = newNode;
21         newNode->next = head;
22         newNode->prev = tail;
23     } else {
24         newNode->next = head;
25         newNode->prev = tail;
26         head->prev = newNode;
```

```

27         tail->next = newNode;
28         head = newNode;
29     }
30     size++;
31 }
32
33 void DoubleLinkedList::add_back(int val) {
34     Node* newNode = new Node;
35     newNode->data = val;
36
37     if (is_empty()) {
38         head = tail = newNode;
39         newNode->next = head;
40         newNode->prev = tail;
41     } else {
42         newNode->next = head;
43         newNode->prev = tail;
44         tail->next = newNode;
45         head->prev = newNode;
46         tail = newNode;
47     }
48     size++;
49 }
50
51 void DoubleLinkedList::add_idx(int val, int idx) {
52     if (idx < 0 || idx > size) throw
std::out_of_range("Index tidak valid");
53     if (idx == 0) {
54         add_front(val);
55     } else if (idx == size) {
56         add_back(val);
57     } else {
58         Node* curr = head;
59         for (int i = 0; i < idx; i++) {
60             curr = curr->next;
61         }
62
63         Node* newNode = new Node;
64         newNode->data = val;
65
66         newNode->prev = curr->prev;
67         newNode->next = curr;
68
69         curr->prev->next = newNode;
70         curr->prev = newNode;
71         size++;
72     }

```

```

73 }
74
75 void DoubleLinkedList::delete_front() {
76     if (is_empty()) return;
77
78     if (size == 1) {
79         delete head;
80         head = tail = nullptr;
81     } else {
82         Node* temp = head;
83         head = head->next;
84         head->prev = tail;
85         tail->next = head;
86         delete temp;
87     }
88     size--;
89 }
90
91 void DoubleLinkedList::delete_back() {
92     if (is_empty()) return;
93
94     if (size == 1) {
95         delete tail;
96         head = tail = nullptr;
97     } else {
98         Node* temp = tail;
99         tail = tail->prev;
100        tail->next = head;
101        head->prev = tail;
102        delete temp;
103    }
104    size--;
105 }
106
107 void DoubleLinkedList::delete_idx(int idx) {
108     if (idx < 0 || idx >= size) throw
std::out_of_range("Index tidak valid");
109     if (idx == 0) {
110         delete_front();
111     } else if (idx == size - 1) {
112         delete_back();
113     } else {
114         Node* curr = head;
115         // Jalan ke target
116         for (int i = 0; i < idx; i++) {
117             curr = curr->next;
118         }

```

```

119         curr->prev->next = curr->next;
120         curr->next->prev = curr->prev;
121
122         delete curr;
123         size--;
124     }
125 }
126
127 void DoubleLinkedList::display() {
128     if (is_empty()) {
129         std::cout << "EMPTY\n";
130         return;
131     }
132     Node* curr = head;
133     do {
134         std::cout << curr->data;
135         curr = curr->next;
136     } while (curr != head);
137     std::cout << "\n";
138 }
139
140 char DoubleLinkedList::get(int idx) {
141     if (idx < 0 || idx >= size) throw
std::out_of_range("Index tidak valid");
142     Node* curr = head;
143     for (int i = 0; i < idx; i++) {
144         curr = curr->next;
145     }
146     return curr->data;
147 }
148
149 void DoubleLinkedList::set(char val, int idx) {
150     if (idx < 0 || idx >= size) throw
std::out_of_range("Index tidak valid");
151     Node* curr = head;
152     for (int i = 0; i < idx; i++) {
153         curr = curr->next;
154     }
155     curr->data = val;
156 }
157
158 void DoubleLinkedList::clear() {
159     while (!is_empty()) {
160         delete_front();
161     }
162 }

```

B. Pembahasan

1. `Init()`

berfungsi untuk menetapkan nilai awal struktur data. Kompleksitas waktu $O(1)$ karena hanya mengatur variabel dasar.

2. `is_empty()`

berfungsi untuk mengecek apakah list tersebut kosong dengan memvalidasi apakah variabel `size` bernilai 0. Kompleksitas Waktu $O(1)$, karena hanya mengecek satu variable.

3. `add_front()`

berfungsi untuk menambahkan data di posisi paling depan. Jika list masih kosong, node akan menjadi head sekaligus tail karena circular. Kompleksitas Waktu $O(1)$ karena tidak ada perulangan.

4. `add_back()`

berfungsi untuk menambahkan node di bagian belakang atau ekor list dengan cara membuat node baru dan mengisi nilainya lalu memperbarui variabel `tail` ke node paling belakang yang baru. Kompleksitas Waktu tetap $O(1)$ karena satu kondisi `if` tidak membatalkan status kompleksitas konstan $O(1)$ dimana Status $O(1)$ baru akan gugur kalau ada perulangan.

5. `add_idx()`

berfungsi untuk memuat posisi data. Setelah memastikan indeks tidak `out_of_range`, algoritma akan mendelegasikan ke `add_front` jika indeks 0, atau `add_back` jika indeks di ujung. Jika di tengah, program melakukan perulangan mencari letak node target (`curr`).

Kenapa valid & benar: Panah node baru diikat terlebih dahulu ke tetangga-tetangganya sebelum tali tetangga diputus dan diarahkan ke node baru. Ini menjaga keutuhan data. Kompleksitas waktu $O(N)$ karena butuh perulangan `for`.

6. `delete_front()`

berfungsi untuk menghapus node pertama guna menghindari memory leak. Jika hanya ada 1 node, list langsung di-reset ke `nullptr`. Jika banyak, head lama disimpan ke variabel `temp`. Setelah aman, `delete temp`; dipanggil untuk menghancurkan memori secara manual dari Heap. Kompleksitas Waktu $O(1)$.

7. `delete_back()`

berfungsi untuk menghapus node ekor guna menghindari memory leak tanpa perlu melakukan perulangan looping dari depan tail lama yang ditampung di `temp` dan dihancurkan ke dengan `delete temp`; Kompleksitas Waktu $O(1)$.

8. `delete_idx()`

berfungsi untuk menghapus node di untul menghindari memory leak. Setelah perulangan berjalan mencari letak dan menyambungkan node di kiri target `curr->prev->next` dengan node di kanan target `curr->next`. Setelah aman, barulah delete `curr`;. Kompleksitas Waktu $O(N)$ karena looping.

9. `display()`

berfungsi untuk mencetak semua isi node ke layar. Putaran akan berhenti ketika sudah menyentuh head. Kompleksitas Waktu $O(N)$.

10. `get()`

berfungsi untuk mengambil atau membaca nilai pada posisi indeks tertentu tanpa menghapusnya dari list. Program menggunakan perulangan for untuk menyusuri list langkah demi langkah. Setelah berjalan sebanyak `idx` dan `curr` berhenti tepat di node yang diminta, fungsi langsung mengembalikan nilai dari node tersebut ke program utama dengan command `return curr->data`;

11. `set()`

berfungsi untuk mengubah atau menimpa nilai data pada posisi (indeks) tertentu dengan nilai (`val`) yang baru. Jika indeksnya benar, penunjuk `curr` akan mulai dari `head->next` lalu menyusuri list menggunakan perulangan for sampai menemukan node target. Fungsi tidak mengembalikan nilai, melainkan langsung menimpa data lama di dalam node tersebut dengan data yang baru melalui command `curr->data = val`;

12. `clear()`

berfungsi s untuk menghapus semua elemen guna menghindari memory leak, sampai list benar-benar tak bersisa. Kompleksitas Waktu $O(N)$ karena menghapus node sebanyak jumlah data.

SOAL 4: Si Palui dan Deretan Angka

Si Palui mengamati fenomena aneh di atas sebuah gelanggang melingkar yang berisi N karakter. Terdapat dua proyektor anomali, Alpha (dimulai dari head, bergerak maju menggunakan next) dan Beta (dimulai dari tail, bergerak mundur menggunakan prev)

Palui melakukan observasi selama R ronde. Pada setiap ronde:

1. Proyektor Alpha dipaksa melompat maju sebanyak X langkah. Proyektor Beta dipaksa melompat mundur sebanyak Y langkah (simultan).
2. Tabrakan Singular: Jika Alpha dan Beta mendarat di titik atau karakter yang sama persis secara bersamaan, karakter tersebut akan hancur dan lenyap dari lingkaran dimensi. Jika ini terjadi, Alpha berpindah ke karakter next dari karakter yang hancur, dan Beta berpindah ke karakter prev dari karakter yang hancur.
3. Resonansi Bersebelahan (Adjacent): Jika Alpha dan Beta saling bersebelahan langsung, sifat kuantum dari kedua karakter tersebut saling bertukar (data antara kedua node tersebut di-swap).

Tentukan sisa karakter pada lingkaran dimensi setelah R ronde selesai dieksekusi

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Format Input

N R

C_1 C_2 ... C_N

X_1 Y_1

X_2 Y_2

...

X_R Y_R

Keterangan: C_i adalah satu karakter char tunggal.

Format Output

Karakter yang tersisa, dicetak berurutan mulai dari head hingga ke elemen terakhir, tanpa spasi. Jika dimensi menjadi kosong, cetak EMPTY.

Input	Output
4 2 A B C D 1 1 2 3	ACB
5 3 V W X Y Z 100000 100000 2 2 5 5	VWYZ

A. Source Code

Tabel 4 Source code prob_b.cpp

```
1 #include <iostream>
2 #include "queue.h"
3
4 using namespace std;
5
6 int main() {
7     int n, k;
8     if (!(cin >> n >> k)) return 0;
9     Queue window;
10    init(&window);
11    long long current_sum = 0;
12    int temp;
13
14    for (int i = 0; i < k; i++) {
15        cin >> temp;
16        enqueue(&window, temp);
17        current_sum += temp;
18    }
19    cout << current_sum;
20
21    for (int i = k; i < n; i++) {
22        cin >> temp;
23        current_sum -= front(&window);
24        ~ dequeue(&window);
25        enqueue(&window, temp);
26        current_sum += temp;
27        cout << " " << current_sum;
```


28	}
29	cout << endl;
30	return 0;
31	}

B. Output Program

```

PS C:\Users\Legion PC\Documents\kuliah\algo\module-3-linked-list-jyaritta> .\prob_b.exe
4 2
A B C D
1 1
2 3
ACB
PS C:\Users\Legion PC\Documents\kuliah\algo\module-3-linked-list-jyaritta> .\prob_b.exe
5 3
V W X Y Z
100000 100000
2 2
5 5
VWYZ

```

Gambar 2 Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Pada awal program, `cin >> N >> R` digunakan untuk membaca dua input dari pengguna, yaitu `N` sebagai jumlah total karakter dan `R` sebagai jumlah observasi. Pengecekan `if (!(cin >> n >> k)) return 0;` berfungsi untuk menghentikan program apabila input yang dimasukkan kosong atau tidak valid. Setelah itu, `arena.init();` untuk menyiapkan struktur Double Linked List.

Selanjutnya, program meminta input data karakter melalui perulangan `for (int i = 0; i < N; i++)`. Setiap karakter yang dibaca ditambahkan ke bagian belakang list menggunakan fungsi `arena.add_back(c)`. Program menetapkan posisi awal proyektor dimana `Node* alpha = arena.head;` dimulai dari depan dan `Node* beta = arena.tail;` yang dimulai dari belakang.

Selama jumlah `R` belum habis, seluruh logika pergerakan proyektor akan terus dieksekusi berulang kali di dalam `for (int i = 0; i < R; i++)`. Program membaca input jarak lompatan `X` dan `Y`. program memangkas jumlah lompatan menggunakan rumus Modulo `move_alpha = X % arena.size;` dan `move_beta = Y % arena.size;`. Alpha kemudian bergerak maju menggunakan `alpha = alpha->next`, sementara Beta bergerak mundur menggunakan `beta = beta->prev`.

Berikutnya, program mengecek atura. Jika `alpha == beta` atau berada di node yang sama, program akan mencatat pijakan proyektor selanjutnya. Jika tidak bertabrakan, program mengecek dengan kondisi `else if (alpha->next == beta || beta->next == alpha)`. Jika bersebelahan, muatan data dari kedua node tersebut saling ditukar (swap) menggunakan variabel temp tanpa menghancurkan node-nya.

Ketika perulangan ronde selesai, program akan menampilkan seluruh sisa karakter dari head hingga akhir menggunakan fungsi `arena.display()`. Terakhir, fungsi `arena.clear()` ; untuk menghancurkan sisa node di dalam memori agar terhindar dari memory leak.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-3-linked-list-jyaritta>